

Programación Avanzada - Sección 1

Ayudantía 2: Arreglos y Funciones

Profesor: Vicente Rivera

Ayudantes: Diego Banda, Martin Ramos



Contacto

Ayudante Cátedra



diego.banda@mail.udp.cl



darkclouds



Click aquí!!

Ayudante Corrector



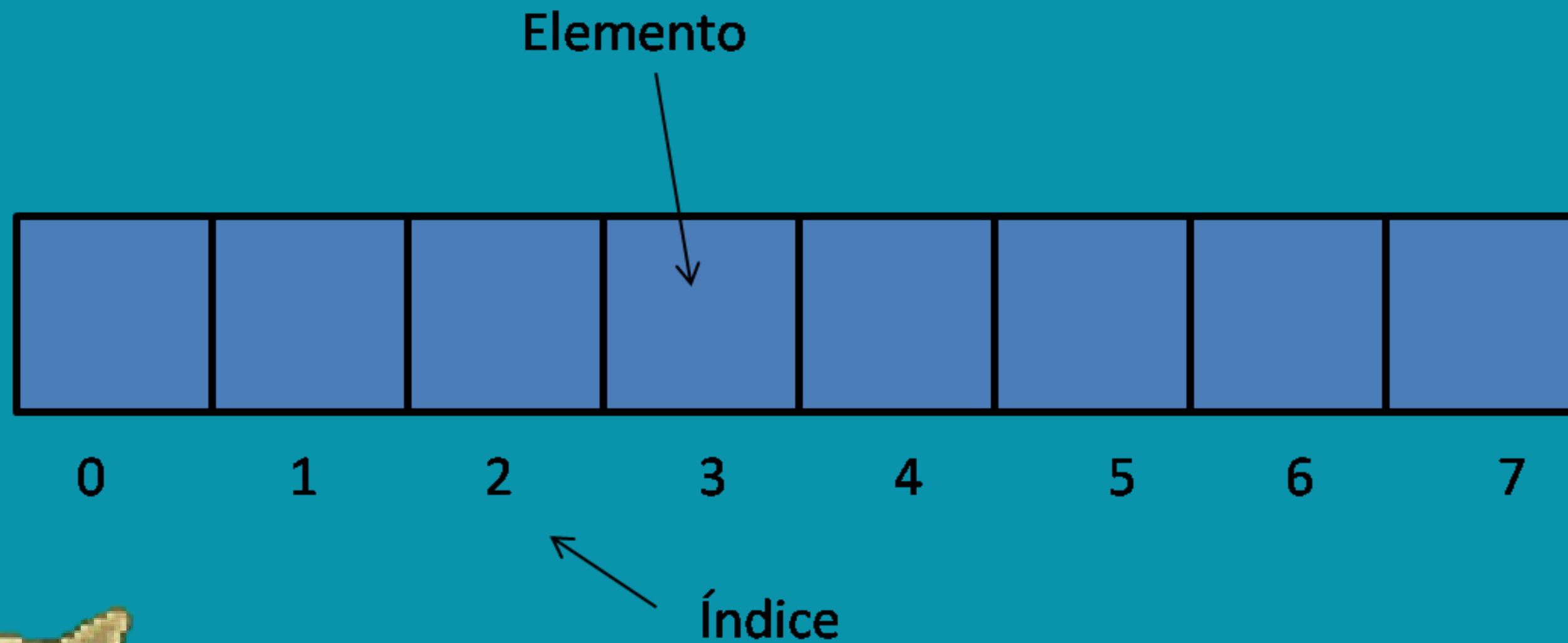
martin.ramos1@mail.udp.cl



martinxikito



Qué es un arreglo



Funciones importantes

- **Inicialización:** `int[] nombre = new int[cantidad];`
- **Inicialización literal:** `int[] nombre = {1, 2, 3, 4};`
- **Acceso:** `nombre[posición]` (ejemplo: `nombre[3]`)
- **Modificación:** `nombre[posición] = x;`
- **Obtener tamaño:** `nombre.length;`



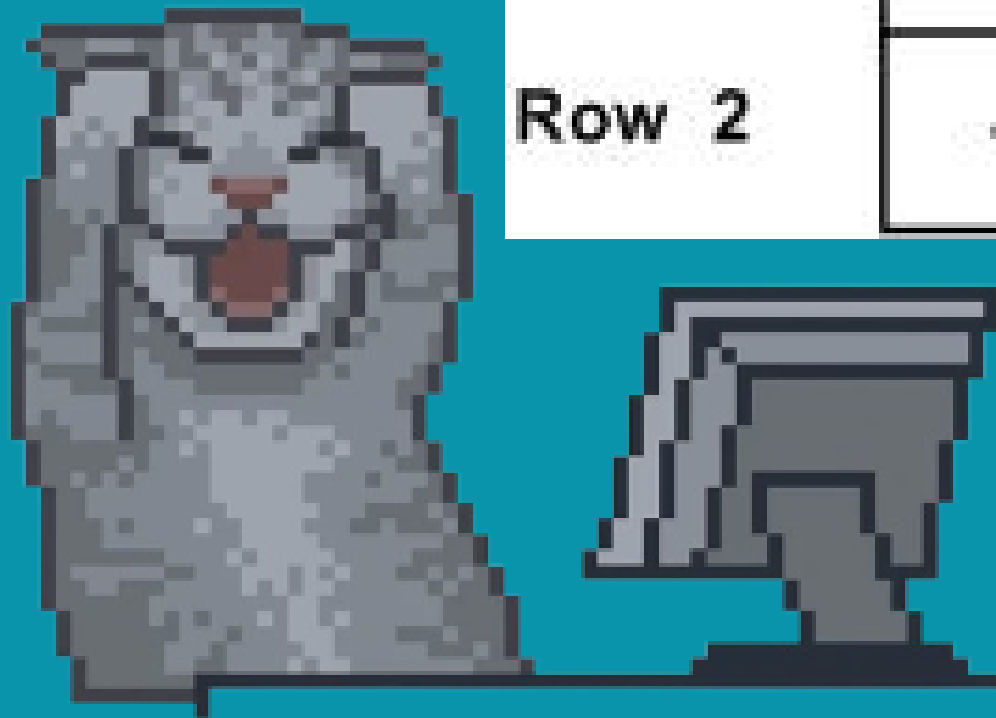
Cómo recorrerlo



```
1  public class main {  
2      public static void main(String[] args) {  
3          int[] arr = {1, 2, 3, 4, 5};  
4          for(int i = 0 ; i < arr.length ; i++){  
5              System.out.printf("%d", arr[i]);  
6          }  
7      }  
8  }
```

Arreglos 2D

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]



Funciones importantes

- **Inicialización:** `int[][] nombre = new int[cantidad1][cantidad2];`
- **Inicialización literal:** `int[] nombre = {{1, 2}, {3, 4}, {5, 6}};`
- **Acceso:** `nombre[posición1][posición2]` (ejemplo: `nombre[3][1]`)
- **Modificación:** `nombre[posición1][posición2] = x;`
- **Obtener tamaño de filas:** `nombre.length;`
- **Obtener tamaño de columnas:** `nombre[posición1].length`



Funciones

Una función es un bloque de código al que se le puede (o no) entregar un input y obtener un output, se puede llamar múltiples veces y normalmente recomendado para bloques de códigos o cálculos que se deben hacer repetidamente

```
1  public class Main{  
2      public static void saludar(){  
3          System.out.printf("Hola!! soy java\n");  
4      }  
5      public static void main(String[] args){  
6          // Oh, quiero que mi programa salude:  
7          saludar();  
8      }  
9  }
```



Funciones

Tipo de dato: Es el tipo de dato que debe retornar la función, puede ser: int, float, double, char, String...

public static: Diferencia el uso que tiene la función dentro de la clase, de momento se utilizará solo esta variante.

```
1 public class Main{  
2     public static int sumar(int a, int b){  
3         return a + b;  
4     }  
5     public static void main(String[] args){  
6         int x = 5;  
7         int y = 3;  
8         System.out.printf("%d", sumar(x, y));  
9     }  
10 }
```

Nombre de la función a través del cual se invocará

Se invoca a través del nombre de la función + las variables requeridas



Output: `> java Main.java`
8

Funciones

Se puede tener una función con el mismo nombre pero distintos parámetros, el código sabrá diferenciar cuál usar automáticamente.

```
1  public class Main{
2      public static int sumar(int a, int b){
3          return a + b;
4      }
5      public static int sumar(int a, int b, int c){
6          return a + b + c;
7      }
8      public static void main(String[] args){
9          int x = 5;
10         int y = 3;
11         int z = 10;
12         System.out.printf("%d\n", sumar(x, y));
13         System.out.printf("%d\n", sumar(x, y, z));
14     }
15 }
```



Sumar y sumar

Output:

```
> java Main.java
8
18
```

Funciones: Paso por referencia

Cuando se manejan arreglos de tipos primitivos, el programa utiliza pasos por referencia para no duplicar memoria innecesariamente.

```
1  public class Main{
2      public static void todoPar(int[] arr){
3          for(int i = 0 ; i < arr.length ; i++){
4              if(arr[i] % 2 != 0){
5                  arr[i] = 0;
6              }
7          }
8      }
9      public static void main(String[] args){
10         int[] arr = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
11         System.out.printf("Arreglo: ");
12         for(int i = 0 ; i < arr.length ; i++){
13             System.out.printf("%d ", arr[i]);
14         }
15         todoPar(arr);
16         System.out.printf("\nArreglo: ");
17         for(int i = 0 ; i < arr.length ; i++){
18             System.out.printf("%d ", arr[i]);
19         }
20         System.out.printf("\n");
21     }
22 }
```

Output:

```
> java Main.java
Arreglo: 1 2 3 4 5 6 7 8 9 10
Arreglo: 0 2 0 4 0 6 0 8 0 10
```



Ejercicio 1

El profesor pokemón oak (el profe roble pa' los amigos) tiene una lista en su pokedex con todos los pokémons registrados, pero olvidó agregar a su programa **una función** para determinar cuales le faltan. Es por esto, que se le pide a usted programar en Java (lenguaje de programación favorito del roble) un método que reciba un arreglo con todos los pokémons encontrados en orden (1, 2, 3...), y si hay un número faltante es porque no se ha registrado tal pokemón y se debe imprimir en pantalla.

```
int[] Pokedex =  
{1,2,3,4,5,6,7,8,9,10,12,13,14,15,16,17,18,19,20,  
22,23,24,25,26,27,28,29,30,31,32,33,34,35  
,37,38,39,40,41,42,43,44,45,46,47,48,49,5  
0,51,52,53,54,55,56,57,58,59,60,61,62,63,64  
,65,66,67,68,69,70,71,72,73,74,75,77,79,82,8  
3,84,85,86,87,88,89,90,91,92,93,94,95,96,97,  
98,99,100,101,102,103,104,105,106,107,108,109,  
110,111,112,113,114,115,116,117,118,119,120,121,122,1  
23,124,125,126,128,129,130,131,132,133,134,13  
5,136,137,139,140,141,142,143,144,148, 149};
```

Ejercicio 2

Batman patrulla Gotham en una cuadrícula 6x6. Cada celda tiene un valor que indica la cantidad de crímenes (Ingresa el valor el usuario). Encuentra la celda con mayor cantidad de crímenes y luego obtén la dirección de la celda para así dársela a batman y que baje la cantidad de crímenes en 5 unidades (Si la cantidad es menor a 5, la dejara en 0).

PD= Asuma que solo habrá una celda con la mayor cantidad de crímenes



Ejercicio 3

A cierto personaje (que ayuda a la sección 1 de progra avanzada con ejercicios), cuando chico amaba el videojuego de ps2 "Dragon Ball: Budokai Tenkaichi 3", pero hoy de grande se dio cuenta que muchos de los personajes no son más que relleno para el juego y la gran mayoría son copias de otros personajes. Es por esto, que entonces le pide ayuda a usted con un programa que elimine a los personajes repetidos del roster (mismo número), pero cómo es un estudiante aún, no tiene mucho poder de computo para duplicar un arreglo, por lo cual, se debe realizar en una **función void**.

- roster = {1, 2, 2, 3, 4, 5, 6, 6, 6, 7, 7, 7, 8, 9, 10, 10, 10, 11, 12, 13, 13, 13, 14, 15, 15, 15}



Muchas gracias!

